

Funciones para Leer Archivos

Malak Sanchez

Monitorias Sistemas Operativos

18 de septiembre de 2026

1. Motivación

Los datos dentro de un archivo de texto pueden estar distribuidos de diferentes maneras (caracteres repetidos, líneas de configuración, campos separados de datos), exigiendo distintas maneras especializadas que nos permitan importarlos con alta precisión en C.

2. Idea Fundamental

El lenguaje C nos ofrece herramientas nativas y funciones de alto nivel para extraer desde el final hacia el principio byte a byte (`fgetc`), extraer la redacción y cadenas masivas separadas por bloques de líneas (`fgets`), y la de encontrar variables bajo estrictos patrones predecibles ignorando espaciados en blanco (`fscanf`).

3. Sintaxis Básica

A continuación, una tabla comparativa con las principales funciones para extraer texto:

Función	¿Qué lee?	Caso de uso ideal
<code>fgetc</code>	Un solo carácter a la vez.	Contar letras específicas o parseo carácter por carácter.
<code>fgets</code>	Una cadena de texto (línea) hasta un salto de línea o límite.	Leer líneas completas de forma segura en archivos de configuración o <i>logs</i> .
<code>fscanf</code>	Datos formateados con un patrón fijo, ignorando espacios en blanco.	Leer registros tabulares, bases de datos simples u organizar numéricos.

```
int char_read = fgetc(file_ptr);
char *text_read = fgets(buffer, limited_size, file_ptr);
int data_matched = fscanf(file_ptr, "%d %s", &int_var, str_var);
```

4. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv){
4     if(argc < 2){
5         printf("Usage: %s <filename>\n", argv[0]);
6         return 1;
7     }
8     FILE *file = fopen(argv[1], "r");
9     if(file == NULL) return 1;
10
11     char buffer[100];
12
13     if(fgets(buffer, sizeof(buffer), file) != NULL){
14         printf("Primera linea: %s", buffer);
15     }
16
17     fclose(file);
18     return 0;
19 }
```

5. Explicación Paso a Paso

1. Verificamos que el usuario haya proporcionado un nombre de archivo al ejecutar el programa.
2. Accedemos a la primera posición del vector de argumentos `argv[1]`.
3. Procedemos configurando el modo de ejecución llamando la apertura del puntero con modo lectura ("`r`").
4. Ocupamos un tamaño límite en una variable estática tipo `char` en forma de contenedor *buffer* local.
5. Para su primera y mínima prueba acudimos a la orden más sana `fgets`, condicionando a la función a utilizar un límite (`sizeof(buffer)`) garantizando que evite corromper otras regiones colindantes de la RAM.
6. Si obtuvo los datos con éxito, se carga automáticamente el resultado y tras la presentación se procede a la orden final `fclose`.

6. Qué ocurre en memoria

El sistema operativo, al inicializar los flujos abiertos de archivos, registra su propio *file descriptor* manteniendo un apuntador al cursor interno. A medida que programamos las llamadas `fgetc`, `fgets` o `fscanf`, se encarga progresivamente de emular cómo el cursor transita los bytes internos en el disco subyacente. Acto seguido sincroniza el volcado parcial hacia a un contenedor local guardable de nuestro programa.

7. Patrones comunes de uso

7.1. Lectura secuencial

Para recorrer dinámicamente el documento completo sin importar qué tan largo sea, usando nuestra bandera de fin *EOF*:

```
1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      if(argc < 2){
5          printf("Usage: %s <filename>\n", argv[0]);
6          return 1;
7      }
8      FILE *file = fopen(argv[1], "r");
9      if (file == NULL) return 1;
10
11     int current_char;
12     while ((current_char = fgetc(file)) != EOF) {
13         printf("%c", (char)current_char);
14     }
15
16     fclose(file);
17     return 0;
18 }
```

8. Medir un archivo manipulando el cursor dinámicamente

Cuando no sabemos la cantidad de elementos o bytes totales que nos esperan dentro del archivo, requerimos manipular el cursor interno usando la función `fseek`. Esta función se apoya en anclas especiales como `SEEK_END` (el final absoluto del archivo) y `SEEK_SET` (el comienzo del mismo).

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char **argv){
5      if(argc < 2){
6          printf("Usage: %s <filename>\n", argv[0]);
7          return 1;
8      }
9      FILE *file = fopen(argv[1], "r");
10     if (file == NULL) return 1;
11
12     // 1. Movemos el cursor al extremo final del archivo
13     fseek(file, 0, SEEK_END);
14
15     // 2. Averiguamos la posicion actual, que equivale al total
16     //    de bytes
17     long file_size = ftell(file);
18     printf("File size: %ld bytes\n", file_size);
19
20     // 3. Regresamos el cursor al principio para reanudar la
21     //    lectura
22     fseek(file, 0, SEEK_SET);
23
24     // Aqui ya podriamos usar 'file_size' de forma segura
25     // en un malloc para contener todos los datos a la vez si
26     // fuese util.
27
28     fclose(file);
29     return 0;
30 }
```

9. Errores Comunes

- **No interceptar el límite de archivo:** Llamar en extremo a lectura en bucles ciegos sin escuchar a las banderas EOF ni la negación de fin NULL, arriesgándose a comportamientos indefinidos de memoria, y basura al infinito.
- **Asignar comodines dispares en `fscanf`:** Pretender recibir el texto dentro de variables declaradas como numéricas como apuntador de tipo `int`, y además, obviar el símbolo o sintaxis imperativa de referenciación (`&`) en las variables comunes.

10. Buenas Prácticas

- Siempre priorizar una carga basada en líneas con **fgets** en el parsing o búsqueda de configuraciones textuales (o *log*), antes que utilizar un sistema vulnerable como pudiese serlo un simple **scanf** genérico.
- Cotejar de nuevo como sistema defensivo la cantidad de conversiones resultantes dadas en respuesta directa de iterar la propia función **fscanf**. Ésta retornará siempre el idílico recuento de emparejamientos confirmados.

11. Ejercicios

1. Genere una rutina basada en **fgetc** que recorra carácter a carácter un archivo de texto y calcule el porcentaje de dígitos respecto del total de caracteres leídos.
2. Construya un programa basado en **fscanf** que lea registros con el formato **<codigo> <grupo> <nota>** y calcule el promedio de notas de cada grupo.